



**INSTITUTO DE EDUCACIÓN SECUNDARIA LA MOLA
NOVELDA**

DESARROLLO DE APLICACIONES WEB

Curso Académico 2025/2026

Desarrollo Web en Entorno Cliente

U02_A02_Ejercicio – Prof. Emilio Ramírez

Autor: Segura Abad, Mario

Fecha: 5 de Octubre, 2025

ÍNDICE

INTRODUCCIÓN	3
ENUNCIADO	4
Función recursiva	4
Función recursiva, 2ª versión	5
DESARROLLO DE LA PRACTICA	7
Descripción	7
Contenido del archivo	7
• Ejercicio 1 - Función recursiva Fibonacci.....	7
• Ejercicio 2 - Función iterativa + medición de rendimiento	7
Cómo visualizarlo	8
Requisitos:	8
Pasos:	8
Objetivo de la práctica	9
• Comprender el funcionamiento de funciones recursivas en JavaScript.....	9
• Implementar la sucesión de Fibonacci con enfoque recursivo e iterativo.	9
• Medir el rendimiento de ambas versiones con console.time().	9
• Identificar el punto en el que la recursividad se vuelve ineficiente.	9
• Documentar correctamente cada ejercicio con nombre, fecha y capturas.	9
• Aplicar buenas prácticas de programación y presentación técnica.....	9
• Entrega del proyecto en formato PDF junto con los archivos fuente y pantallazos.....	9
CONCLUSIÓN.....	10

INTRODUCCIÓN

En la “Función recursiva” deberemos, primero de todo, leer la explicación y definición del término recursividad descrita en la propia tarea. Una vez sepamos y entendamos dicha explicación crearemos una función recursiva que muestre la sucesión de Fibonacci y comprobar su correcto funcionamiento y visualización en el navegador.

En la “Función recursiva, 2ª versión” utilizaremos las funciones `console.time()` y `console.timeEnd()` e indicaremos a partir de qué número la función tardará más computacionalmente utilizando dichas funciones recursivas.

ENUNCIADO

Función recursiva

La recursividad es la capacidad que tienen las funciones de llamarse a sí mismas. La ventaja de la recursividad es la reducción de la complejidad del problema hasta que se llegue a un punto en el cual ya no haga falta utilizar la recursión, puesto que la resolución es algo directa o simple. Por lo tanto, la recursividad debe parar cuando lleguemos al caso base, y en el resto de los casos el resultado pasa por llamar otra vez a la función recursiva, variando la llamada inicial hasta conseguir llegar al caso base.

Ahora bien, la recursividad es considerada una técnica de programación peligrosa (puede generar llamadas infinitas y desbordar la pila del sistema), pero está considerada una forma muy elegante de resolver problemas. A veces, se presentan problemas complejos de resolver y tras darles un enfoque recursivo aparecen soluciones extraordinarias simples.

Importante: En general, las soluciones recursivas son mucho más costosas computacionalmente que las soluciones iterativas, por lo que se recomienda que se recurra a la recursividad solo en aquellos casos en los que la solución iterativa no sea posible.

Crea una función recursiva que muestre la sucesión de Fibonacci. Comprueba en un navegador que la función visualiza correctamente la secuencia deseada.

La función recursiva se define como $F(n)=F(n-1)+F(n-2)$, con casos base $F(0)=0$ y $F(1)=1$. Esta implementación es elegante y fácil de entender, ya que refleja directamente la relación de recurrencia.

Para nuestros ejercicios pediremos al usuario el valor de n , siendo n mayor o igual a 0.

Resultados:

$$F(2) = 1$$

$$F(3) = 2$$

$$F(4) = 3$$

$$F(5) = 5$$

$$F(6) = 8$$

$$F(7) = 13$$

$$F(8) = 21$$

...

$$F(N) = F(N-1) + F(N-2)$$

Captura de pantalla – [FuncionRecursiva.html](#)

```

<> FuncionRecursiva.html X <> FuncionRecursiva2.html
<> FuncionRecursiva.html > html > body > p
1  <!-- Autor: Mario Segura Abad -->
2  <!-- Fecha: 05/10/2025-->
3
4  <!DOCTYPE html>
5  <html lang="es">
6      <head>
7          <meta charset="UTF-8">
8          <meta name="viewport" content="width=device-width, initial-scale=1.0">
9          <title>U02_A02_Ejercicio - Sucesión de Fibonacci</title>
10     </head>
11     <body>
12         <h2>Sucesión de Fibonacci (Función Recursiva)</h2>
13
14         <label for="n">Introduce un número n (>= 0)</label>
15         <input type="number" id="n" min="0">
16         <button onclick="mostrarFibonacci()">Calcular</button>
17
18         <div id="resultado" style="margin-top: 15px;"></div>
19
20         <script>
21             //Función recursiva de Fibonacci
22             function fibonacci(n) {
23                 if(n === 0) {
24                     return 0; // Caso base 1
25                 } else if(n === 1) {
26                     return 1; // Caso base 2
27                 } else {
28                     // Llamadas recursivas
29                     return fibonacci(n - 1) + fibonacci(n - 2);
30                 }
31             }
32
33             // Función que muestra la secuencia hasta F(n)
34             function mostrarFibonacci() {
35                 const n = parseInt(document.getElementById("n").value);
36                 const resultadoDivision = document.getElementById("resultado");
37
38                 if(isNaN(n) || n < 0) {
39                     resultadoDivision.innerHTML = "<b>Por favor, introduce un número mayor o igual a 0.</b>";
40                     return;
41                 }
42
43                 let salida = "";
44                 for(let i = 0; i <= n; i++) {
45                     salida += `F(${i}) = ${fibonacci(i)}<br>`;
46                 }
47                 resultadoDivision.innerHTML = salida;
48             }
49         </script>
50
51         <!-- Prueba de Autoría -->
52         <p>Autor: Mario Segura Abad<br></p>
53         <p>Fecha de realización: 05/10/2025</p>
54     </body>
55 </html>

```

Función recursiva, 2ª versión

Realizar el ejercicio anterior también de forma iterativa, y utilizando las funciones: `console.time()` y `console.timeEnd()` indicar a partir de qué número la función fibonacci tarda mucho más computacionalmente usando funciones recursivas.

Captura de pantalla – FuncionRecursiva2.html

```

< FuncionRecursiva.html > FuncionRecursiva2.html X
< FuncionRecursiva2.html > html > body > p
1 <!-- Autor: Mario Segura Abad -->
2 <!-- Fecha: 05/10/2025 -->
3
4 <!DOCTYPE html>
5 <html lang="es">
6 <head>
7 <meta charset="UTF-8">
8 <meta name="viewport" content="width=device-width, initial-scale=1.0">
9 <title>Comparativa Fibonacci Recursivo vs Iterativo</title>
10 </head>
11 <body>
12 <h2>Fibonacci: Recursivo vs Iterativo</h2>
13
14 <label for="n">Introduce un número n (>= 0, recursivo limitado a 30): </label>
15 <input type="number" id="n" min="0">
16 <button onclick="compararFibonacci()">Calcular y comparar</button>
17
18 <div id="resultado" style="margin-top: 15px;"></div>
19
20 <script>
21 // Función recursiva
22 function fibonacciRecursivo(n) {
23   if (n === 0) return 0;
24   if (n === 1) return 1;
25   return fibonacciRecursivo(n - 1) + fibonacciRecursivo(n - 2);
26 }
27
28 // Función iterativa
29 function fibonacciIterativo(n) {
30   if (n === 0) return 0;
31   if (n === 1) return 1;
32
33   let a = 0, b = 1;
34   for (let i = 2; i <= n; i++) {
35     [a, b] = [b, a + b];
36   }
37   return b;
38 }
39
40 function compararFibonacci() {
41   const n = parseInt(document.getElementById("n").value);
42   const resultadoDivision = document.getElementById("resultado");
43
44   if (isNaN(n) || n < 0) {
45     resultadoDivision.innerHTML = "<b>Introduce un número mayor o igual a 0.</b>";
46     return;
47   }
48
49   let salida = "";
50
51 // Recursivo solo hasta n = 30 para no colgar el navegador
52 if (n <= 30) {
53   const t0 = performance.now();
54   const recursivo = fibonacciRecursivo(n);
55   const t1 = performance.now();
56   salida += `<p><b>F(${n}) Recursivo:</b> ${recursivo} (Tiempo: ${(t1-t0).toFixed(3)} ms)</p>`;
57 } else {
58   salida += `<p>F(${n}) Recursivo: Muy grande para calcular recursivamente</p>`;
59 }
60
61 // Iterativo
62 const t2 = performance.now();
63 const iterativo = fibonacciIterativo(n);
64 const t3 = performance.now();
65 salida += `<p><b>F(${n}) Iterativo:</b> ${iterativo} (Tiempo: ${(t3-t2).toFixed(3)} ms)</p>`;
66
67 resultadoDiv.innerHTML = salida;
68 }
69 </script>
70
71 <!-- Prueba de Autoría -->
72 <p>Autor: Mario Segura Abad<br></p>
73 <p>Fecha de realización: 05/10/2025</p>
74 </body>
75 </html>

```

DESARROLLO DE LA PRACTICA

Práctica DWEK – Unidad 2

Descripción

Este proyecto contiene los ejercicios prácticos relacionados con el uso de funciones recursivas e iterativas en JavaScript, aplicados a la generación de la sucesión de Fibonacci. El objetivo principal es comprender el funcionamiento de la recursividad, identificar sus ventajas y limitaciones, y compararla con una solución iterativa más eficiente.

Se han desarrollado dos versiones de la función `fibonacci`: una recursiva y otra iterativa. Además, se ha utilizado `console.time()` y `console.timeEnd()` para medir el rendimiento de ambas versiones y determinar a partir de qué valor de `n` la recursividad se vuelve computacionalmente costosa.

Contenido del archivo

- **Ejercicio 1 - Función recursiva Fibonacci**

Se ha creado una función recursiva que calcula el valor de Fibonacci para un número `n` introducido por el usuario. La función se basa en la relación:

$F(n) = F(n-1) + F(n-2)$, con casos base **$F(0) = 0$** y **$F(1) = 1$** .

Ejemplo de ejecución:

Sucesión de Fibonacci (Función Recursiva)

Introduce un número `n` (≥ 0)

$F(0) = 0$

$F(1) = 1$

$F(2) = 1$

$F(3) = 2$

$F(4) = 3$

$F(5) = 5$

Autor: Mario Segura Abad

Fecha de realización: 05/10/2025

- **Ejercicio 2 - Función iterativa + medición de rendimiento**

Se ha implementado una versión iterativa de la función `fibonacci`, mucho más eficiente en términos de tiempos de ejecución.

Se ha utilizado `console.time()` y `console.timeEnd()` para comparar el rendimiento entre ambas versiones.

Ejemplo de ejecución:

Fibonacci Recursivo e Iterativo

Introduce un número n (≥ 0):

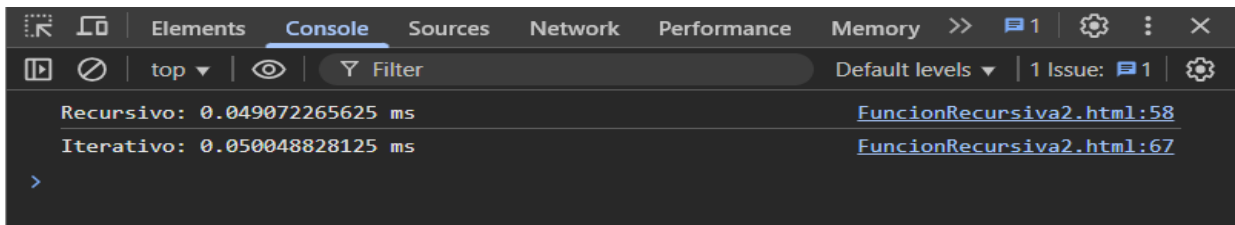
F(10) recursivo: 55

F(10) iterativo: 55

Consulta la consola (F12) para ver los tiempos.

Autor: Mario Segura Abad

Fecha de realización: 05/10/2025



Cómo visualizarlo

Requisitos:

- Editor de código: Visual Studio Code, en mi caso.
- Navegador web para scripts con `prompt()` y `document.write()`.
- Node.js (opcional) para ejecutar scripts con `console.log()` y medir tiempos.

Pasos:

1. Abrir los archivos `.js` o `.html` en el editor.
2. Ejecutar los scripts en navegador o consola según el tipo.
3. Introducir el valor de n cuando se solicite.
4. Observar el resultado en pantalla o consola.
5. Revisar las capturas de ejecución incluidas en la carpeta correspondiente.

Objetivo de la práctica

- Comprender el funcionamiento de funciones recursivas en JavaScript.
- Implementar la sucesión de Fibonacci con enfoque recursivo e iterativo.
- Medir el rendimiento de ambas versiones con `console.time()`.
- Identificar el punto en el que la recursividad se vuelve ineficiente.
- Documentar correctamente cada ejercicio con nombre, fecha y capturas.
- Aplicar buenas prácticas de programación y presentación técnica.
- Entrega del proyecto en formato PDF junto con los archivos fuente y pantallazos.

CONCLUSIÓN

Esta práctica ha permitido entender la recursividad como técnica elegante pero costosa, especialmente en problemas como Fibonacci.

La comparación con la versión iterativa ha demostrado que, a partir de valores como $n = 35$, la recursión se vuelve notablemente más lenta.

Además, se ha reforzado la importancia de medir el rendimiento del código y documentar correctamente cada ejercicio.